

4. Modelo UML

El modelo UML de UTEQMarket describe la estructura estática del sistema mediante clases, atributos, operaciones, enumeraciones y relaciones. El modelo fue elaborado en PowerDesigner a partir de los requisitos funcionales, reglas de negocio y entidades identificadas para la plataforma. La presente sección documenta el Diagrama de Clases y su correspondencia con el modelo físico implementado en Microsoft SQL Server.

4.1 Diagrama de clases

El Diagrama de Clases representa los principales elementos del dominio de UTEQMarket y la forma en que colaboran entre sí. El modelo está compuesto por las clases Usuario, Publicacion, Categoria, ImagenPublicacion, Favorito, Contacto, Calificacion y Reporte. Además, utiliza las enumeraciones RolUsuario, EstadoPublicacion, EstadoReporte y MotivoReporte para restringir los valores permitidos en determinados atributos.

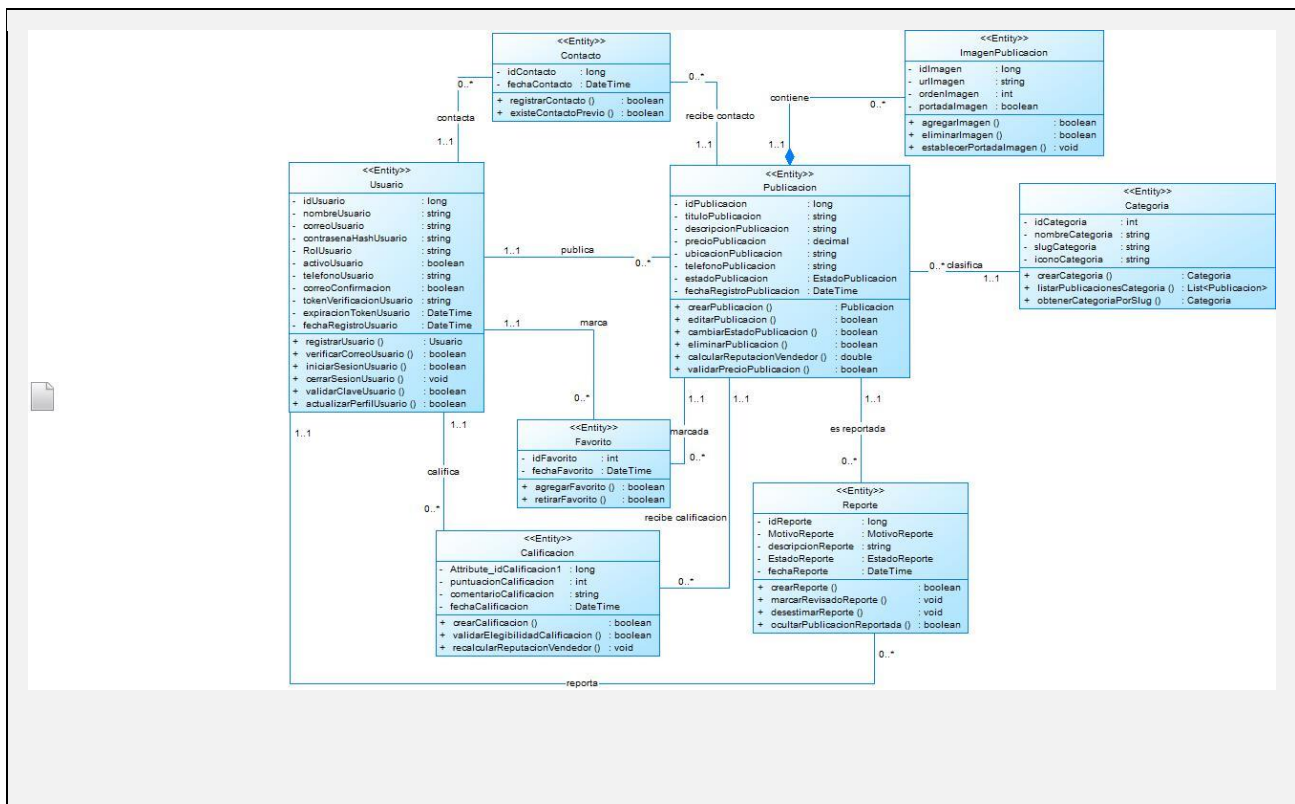


Figura 4.1. Diagrama de Clases UML final de UTEQMarket.

Fuente: Elaboración propia mediante PowerDesigner o SQL Server, según corresponda.

Explicación del diagrama

La clase Usuario representa a los miembros de la comunidad universitaria que utilizan la plataforma. Mantiene la información de la cuenta, correo institucional, contraseña, teléfono, rol, estado de activación y datos de verificación. Sus operaciones permiten registrar usuarios, verificar el correo, iniciar y cerrar sesión, validar la contraseña y actualizar el perfil.

La clase Publicacion representa los productos o servicios ofrecidos en la plataforma. Contiene el título, descripción, precio, ubicación, teléfono de contacto, estado y fecha de registro. Sus operaciones permiten crear, modificar, eliminar y cambiar el estado de una publicación, además de validar el precio y calcular la reputación del vendedor.

La clase Categoria organiza las publicaciones según el tipo de producto o servicio. ImagenPublicacion almacena las imágenes asociadas a una publicación, su orden de presentación y la indicación de portada. Favorito, Contacto, Calificacion y Reporte registran las diferentes interacciones realizadas por los usuarios sobre las publicaciones.

Relaciones, herencia y tipos de vínculo

Las relaciones Usuario–Publicacion y Categoria–Publicacion son asociaciones de uno a muchos. Las clases Favorito, Contacto, Calificacion y Reporte funcionan como clases asociativas que resuelven conceptualmente las relaciones muchos a muchos entre Usuario y Publicacion mediante dos asociaciones de uno a muchos.

La relación entre Publicacion e ImagenPublicacion se modela mediante composición, porque la imagen depende del ciclo de vida de la publicación. El modelo no utiliza agregación ni generalización. Los roles comprador, vendedor y administrador se representan mediante la enumeración RolUsuario, evitando crear subclases que dupliquen atributos y operaciones.

4.2 Descripción de clases

Las clases del Diagrama de Clases distribuyen las responsabilidades del sistema de forma coherente. La Tabla 4.1 resume la responsabilidad principal de cada clase.

Tabla 4.1. Descripción de las clases de UTEQMarket.

Clase	Responsabilidad
-------	-----------------

Clase	Responsabilidad
Usuario	Representar a los miembros de la comunidad UTEQ. Administra la cuenta, autenticación, verificación del correo, rol, estado y datos del perfil.
Publicacion	Representar los productos o servicios ofrecidos. Gestiona título, descripción, precio, ubicación, contacto, estado y fecha de registro.
Categoria	Clasificar y organizar las publicaciones según el tipo de producto o servicio ofrecido.
ImagenPublicacion	Gestionar las imágenes asociadas a una publicación, incluyendo su URL, orden y condición de portada.
Favorito	Registrar la relación entre un usuario y una publicación guardada como favorita.
Contacto	Registrar las solicitudes de contacto realizadas por un usuario sobre una publicación.
Calificacion	Registrar puntuaciones y comentarios emitidos por los usuarios, aportando información para la reputación del vendedor.
Reporte	Registrar y administrar incidencias notificadas sobre una publicación para su revisión administrativa.

Fuente: Elaboración propia.

Tabla 4.2. Enumeraciones utilizadas en el modelo.

Enumeración	Responsabilidad
RolUsuario	Define los roles válidos: COMPRADOR, VENDEDOR y ADMINISTRADOR.
EstadoPublicacion	Restringe el estado de una publicación a ACTIVA, PAUSADA o VENDIDA.
EstadoReporte	Define los estados del reporte: PENDIENTE, REVISADO o DESESTIMADO.
MotivoReporte	Define los motivos permitidos para reportar una publicación.

Fuente: Elaboración propia.

4.3 Atributos

Los atributos representan el estado interno de las clases. En el modelo UML se definieron con visibilidad privada, representada mediante el símbolo -, para aplicar el principio de encapsulamiento. Los campos foráneos creados posteriormente en el modelo físico no se incluyen como atributos del Diagrama de Clases cuando la relación ya se encuentra representada mediante una asociación.

Nota técnica. Los identificadores físicos como idUsuarioPublicacion, idCategoriaPublicacion o idPublicacionFavorito pertenecen al PDM y a SQL Server. En el OOM, las relaciones se muestran mediante asociaciones y cardinalidades.

Tabla 4.3. Atributos de las clases de UTEQMarket.

Clase	Atributo	Tipo	Visibilidad	Descripción
Usuario	idUsuario	long	private (-)	Identificador único del usuario.
Usuario	nombreUsuario	string	private (-)	Nombre completo del usuario.
Usuario	correoUsuario	string	private (-)	Correo institucional utilizado para registro y autenticación.
Usuario	contrasenaHashUsuario	string	private (-)	Contraseña almacenada mediante hash.
Usuario	rolUsuario	RolUsuario	private (-)	Rol asignado dentro de la plataforma.
Usuario	activoUsuario	boolean	private (-)	Indica si la cuenta está activa.
Usuario	telefonoUsuario	string	private (-)	Número de teléfono o WhatsApp.
Usuario	correoConfirmacion	boolean	private (-)	Indica si el correo fue confirmado.
Usuario	tokenVerificacionUsuario	string	private (-)	Token temporal para verificar la cuenta.
Usuario	expiracionTokenUsuario	DateTime	private (-)	Fecha y hora de expiración del token.
Usuario	fechaRegistroUsuario	DateTime	private (-)	Fecha y hora de creación de la cuenta.
Publicacion	idPublicacion	long	private (-)	Identificador único de la publicación.
Publicacion	tituloPublicacion	string	private (-)	Título del producto o servicio.
Publicacion	descripcionPublicacion	string	private (-)	Descripción detallada de la publicación.
Publicacion	precioPublicacion	decimal	private (-)	Precio establecido por el vendedor.
Publicacion	ubicacionPublicacion	string	private (-)	Ubicación de referencia.
Publicacion	telefonoPublicacion	string	private (-)	Número de contacto de la publicación.
Publicacion	estadoPublicacion	EstadoPublicacion	private (-)	Estado actual: activa, pausada o vendida.
Publicacion	fechaRegistroPublicacion	DateTime	private (-)	Fecha y hora de creación.
Categoria	idCategoria	int	private (-)	Identificador único de la categoría.
Categoria	nombreCategoria	string	private (-)	Nombre descriptivo de la categoría.
Categoria	slugCategoria	string	private (-)	Identificador textual utilizado en búsquedas o URL.
Categoria	iconoCategoria	string	private (-)	Icono representativo de la categoría.
ImagenPublicacion	idImagen	long	private (-)	Identificador único de la imagen.

Clase	Atributo	Tipo	Visibilidad	Descripción
ImagenPublicacion	urlImagen	string	private (-)	Dirección de almacenamiento de la imagen.
ImagenPublicacion	ordenImagen	int	private (-)	Orden de presentación.
ImagenPublicacion	portadaImagen	boolean	private (-)	Indica si es la imagen de portada.
Favorito	idFavorito	int	private (-)	Identificador único del favorito.
Favorito	fechaFavorito	DateTime	private (-)	Fecha y hora de registro del favorito.
Contacto	idContacto	long	private (-)	Identificador único del contacto.
Contacto	fechaContacto	DateTime	private (-)	Fecha y hora de la solicitud de contacto.
Calificacion	idCalificacion	long	private (-)	Identificador único de la calificación.
Calificacion	puntuacionCalificacion	int	private (-)	Puntuación asignada entre uno y cinco.
Calificacion	comentarioCalificacion	string	private (-)	Comentario de la evaluación.
Calificacion	fechaCalificacion	DateTime	private (-)	Fecha y hora de emisión.
Reporte	idReporte	long	private (-)	Identificador único del reporte.
Reporte	motivoReporte	MotivoReporte	private (-)	Motivo por el que se reporta la publicación.
Reporte	descripcionReporte	string	private (-)	Descripción adicional del reporte.
Reporte	estadoReporte	EstadoReporte	private (-)	Estado actual del proceso de revisión.
Reporte	fechaReporte	DateTime	private (-)	Fecha y hora de creación del reporte.

Fuente: Elaboración propia.

Tabla 4.4. Enumeraciones utilizadas como tipos de atributos.

Enumeración	Atributo que la utiliza	Valores permitidos
RolUsuario	Usuario.rolUsuario	COMPRADOR, VENDEDOR, ADMINISTRADOR
EstadoPublicacion	Publicacion.estadoPublicacion	ACTIVA, PAUSADA, VENDIDA
EstadoReporte	Reporte.estadoReporte	PENDIENTE, REVISADO, DESESTIMADO
MotivoReporte	Reporte.motivoReporte	PUBLICACION_FALSA, CONTENIDO_INAPROPIADO, PRECIO_ENGANOSO, POSIBLE_ESTAFA, OTRO

Fuente: Elaboración propia.

4.4 Métodos

Los métodos representan el comportamiento de las clases. En el Diagrama de Clases se definieron con visibilidad pública, representada por el símbolo +, para permitir su invocación desde otros componentes del sistema.

Tabla 4.5. Métodos de las clases de UTEQMarket.

Clase	Método y parámetros	Retorno	Visibilidad	Descripción
Usuario	registrarUsuario(nombreUsuario: string, correoUsuario: string, claveUsuario: string)	Usuario	public (+)	Registra una nueva cuenta y devuelve el usuario creado.
Usuario	verificarCorreoUsuario(tokenUsuario: string)	boolean	public (+)	Verifica el correo mediante el token asociado.
Usuario	iniciarSesionUsuario(correoUsuario: string, claveUsuario: string)	boolean	public (+)	Valida las credenciales e inicia la sesión.
Usuario	cerrarSesionUsuario()	void	public (+)	Finaliza la sesión activa.
Usuario	validarClaveUsuario(claveUsuario: string)	boolean	public (+)	Comprueba las reglas de seguridad de la contraseña.
Usuario	actualizarPerfilUsuario(datosUsuario: PerfilUsuarioDTO)	boolean	public (+)	Actualiza los datos del perfil.
Publicacion	crearPublicacion(datosPublicacion: PublicacionDTO)	Publicacion	public (+)	Crea una publicación y devuelve el objeto registrado.
Publicacion	editarPublicacion(datosPublicacion: PublicacionDTO)	boolean	public (+)	Modifica una publicación existente.
Publicacion	cambiarEstadoPublicacion(nuevoEstadoPublicacion: EstadoPublicacion)	boolean	public (+)	Cambia el estado de la publicación.
Publicacion	eliminarPublicacion()	boolean	public (+)	Elimina una publicación cuando existe autorización.
Publicacion	calcularReputacionVendedor()	double	public (+)	Calcula la reputación a partir de las calificaciones.
Publicacion	validarPrecioPublicacion(precioPublicacion: decimal)	boolean	public (+)	Comprueba que el precio sea positivo y válido.
Categoria	crearCategoria(nombreCategoria: string, slugCategoria: string, iconoCategoria: string)	Categoria	public (+)	Registra una categoría.
Categoria	listarPublicacionesCategoria()	List<Publicacion>	public (+)	Devuelve las publicaciones de una categoría.
Categoria	obtenerCategoriaPorSlug(slugCategoria: string)	Categoria	public (+)	Busca una categoría mediante su slug.
ImagenPublicacion	agregarImagen(archivoImagen: ArchivoDTO)	boolean	public (+)	Agrega una imagen después de validar el archivo.
ImagenPublicacion	eliminarImagen()	boolean	public (+)	Elimina una imagen asociada.
ImagenPublicacion	establecerPortadaImagen()	void	public (+)	Establece la imagen seleccionada como portada.
Favorito	agregarFavorito(idUsuarioFavorito: long, idPublicacionFavorito: long)	boolean	public (+)	Registra una publicación como favorita.

Clase	Método y parámetros	Retorno	Visibilidad	Descripción
Favorito	retirarFavorito()	boolean	public (+)	Retira una publicación de favoritos.
Contacto	registrarContacto(idUsuarioContacto: long, idPublicacion: long)	boolean	public (+)	Registra una solicitud de contacto.
Contacto	existeContactoPrevio(idUsuario: long, idPublicacion: long)	boolean	public (+)	Comprueba si ya existe un contacto previo.
Calificacion	crearCalificacion(datosCalificacion: CalificacionDTO)	boolean	public (+)	Registra una puntuación y comentario.
Calificacion	validarElegibilidadCalificacion(idUsuarioCalificacion: long, idPublicacionCalificacion: long)	boolean	public (+)	Verifica que el usuario pueda calificar.
Calificacion	recalcularReputacionVendedor(idVendedor: long)	void	public (+)	Actualiza la reputación del vendedor.
Reporte	crearReporte(datosReporte: ReporteDTO)	boolean	public (+)	Registra un reporte sobre una publicación.
Reporte	marcarRevisadoReporte()	void	public (+)	Cambia el estado del reporte a revisado.
Reporte	desestimarReporte()	void	public (+)	Cambia el estado a desestimado.
Reporte	ocultarPublicacionReportada()	boolean	public (+)	Oculta una publicación que incumple las reglas.

Fuente: Elaboración propia.

Tabla 4.6. Significado de los tipos de retorno.

Tipo de retorno	Significado
boolean	Devuelve true cuando la operación se completa y false cuando no puede completarse.
void	Ejecuta una acción sin devolver un valor.
Usuario	Devuelve un objeto de la clase Usuario.
Publicacion	Devuelve un objeto de la clase Publicacion.
Categoria	Devuelve un objeto de la clase Categoria.
List<Publicacion>	Devuelve una colección de publicaciones.
double	Devuelve un valor decimal utilizado para la reputación.

Fuente: Elaboración propia.

4.5 Relaciones

Las relaciones UML describen la manera en que las clases se conectan y colaboran. En UTEQMarket se emplean asociaciones y una composición. La agregación, la dependencia explícita y la generalización no forman parte del diagrama final.

Asociación

La asociación representa un vínculo estructural entre clases. En el diagrama se muestra mediante una línea simple sin punta de flecha, acompañada por el nombre de la relación y sus multiplicidades.

Tabla 4.7. Asociaciones del Diagrama de Clases.

Clase A	Relación	Clase B	Descripción
Usuario	publica	Publicacion	Un usuario puede registrar varias publicaciones; cada publicación pertenece a un usuario.
Categoria	clasifica	Publicacion	Una categoría puede clasificar varias publicaciones; cada publicación pertenece a una categoría.
Usuario	marca	Favorito	Un usuario puede generar varios registros de favorito.
Publicacion	es marcada	Favorito	Una publicación puede aparecer en varios registros de favorito.
Usuario	contacta	Contacto	Un usuario puede generar varias solicitudes de contacto.
Publicacion	recibe contacto	Contacto	Una publicación puede recibir varias solicitudes de contacto.
Usuario	califica	Calificacion	Un usuario puede emitir varias calificaciones.
Publicacion	recibe calificación	Calificacion	Una publicación puede recibir varias calificaciones.
Usuario	reporta	Reporte	Un usuario puede generar varios reportes.
Publicacion	es reportada	Reporte	Una publicación puede recibir varios reportes.

Fuente: Elaboración propia.

Composición

La composición representa una relación fuerte entre un objeto principal y sus partes. En UTEQMarket se utiliza entre Publicacion e ImagenPublicacion. El rombo negro se ubica junto a Publicacion, porque una imagen no posee sentido independiente dentro del sistema.

Publicacion 1..1 ◆———— contiene ————— 0..* ImagenPublicacion

Agregación

El modelo no utiliza agregación. Esta relación se emplearía cuando una parte pudiera existir independientemente del objeto contenedor. En este caso, ImagenPublicacion depende de Publicacion, por lo que corresponde utilizar composición.

Dependencia

No se dibujaron dependencias explícitas. Algunos métodos utilizan objetos auxiliares como PerfilUsuarioDTO, PublicacionDTO, ArchivoDTO, CalificacionDTO y ReporteDTO; esto representa una dependencia conceptual, pero dichas clases no forman parte del alcance del diagrama.

Generalización

No se utiliza generalización o herencia. Los roles comprador, vendedor y administrador no se modelan como subclases de Usuario, sino mediante el atributo rolUsuario de tipo RolUsuario. Esta decisión evita duplicación de datos y comportamiento.

Nota técnica. En el modelo físico, los nombres publica, clasifica, marca, contacta, califica y reporta no se convierten en tablas. Se implementan mediante referencias y claves foráneas entre las ocho tablas principales.

4.6 Cardinalidades

Las cardinalidades o multiplicidades indican la cantidad mínima y máxima de instancias que pueden participar en una relación. PowerDesigner utiliza la notación 1..1 para representar una instancia obligatoria y 0..* para representar cero o muchas instancias.

Tabla 4.8. Significado de las cardinalidades UML.

Cardinalidad	Significado	Interpretación
1 o 1..1	Exactamente uno	La relación es obligatoria y solo admite una instancia.
0..1	Cero o uno	La relación es opcional y admite como máximo una instancia.
1..*	Uno o muchos	Debe existir al menos una instancia y pueden existir varias.
0..*	Cero o muchos	La relación es opcional y puede contener múltiples instancias.
*	Muchos	Cantidad indeterminada; se recomienda especificar el mínimo.

Fuente: Elaboración propia.

Tabla 4.9. Cardinalidades del Diagrama de Clases de UTEQMarket.

Clase A	Relación	Clase B	Cardinalidad	Interpretación
Usuario	publica	Publicacion	1..1 — 0..*	Un usuario puede registrar cero o muchas publicaciones; cada publicación pertenece a un usuario.
Categoria	clasifica	Publicacion	1..1 — 0..*	Una categoría puede clasificar cero o muchas publicaciones; cada publicación pertenece a una categoría.
Publicacion	contiene	ImagenPublicacion	1..1 — 0..*	Una publicación puede contener cero o muchas imágenes; cada imagen pertenece a una publicación.
Usuario	marca	Favorito	1..1 — 0..*	Un usuario puede generar cero o muchos favoritos; cada favorito pertenece a un usuario.
Publicacion	es marcada	Favorito	1..1 — 0..*	Una publicación puede aparecer en cero o muchos favoritos; cada favorito corresponde a una publicación.
Usuario	contacta	Contacto	1..1 — 0..*	Un usuario puede realizar cero o muchos contactos; cada contacto pertenece a un usuario.
Publicacion	recibe contacto	Contacto	1..1 — 0..*	Una publicación puede recibir cero o muchos contactos; cada contacto corresponde a una publicación.
Usuario	califica	Calificacion	1..1 — 0..*	Un usuario puede emitir cero o muchas calificaciones; cada calificación pertenece a un usuario.
Publicacion	recibe calificación	Calificacion	1..1 — 0..*	Una publicación puede recibir cero o muchas calificaciones; cada calificación corresponde a una publicación.
Usuario	reporta	Reporte	1..1 — 0..*	Un usuario puede generar cero o muchos reportes; cada reporte pertenece a un usuario.
Publicacion	es reportada	Reporte	1..1 — 0..*	Una publicación puede recibir cero o muchos reportes; cada reporte corresponde a una publicación.

Fuente: Elaboración propia.

Las multiplicidades 0..1 y 1..* se explican como parte de la notación UML, pero no se utilizan en las once relaciones finales del modelo. En UTEQMarket, el extremo principal se configuró como 1..1 y el extremo dependiente como 0..*.

4.7 Estereotipos utilizados

Los estereotipos amplían el significado de los elementos UML e indican la función que cumple cada clase dentro del modelo. En UTEQMarket se utilizaron los estereotipos <<Entity>> y <<enumeration>>.

Tabla 4.10. Estereotipos utilizados en UTEQMarket.

Estereotipo	Elementos que lo utilizan	Descripción
<<Entity>>	Usuario, Publicacion, Categoria, ImagenPublicacion, Favorito, Contacto, Calificacion y Reporte	Identifica clases persistentes que representan información y comportamiento del dominio.
<<enumeration>>	RolUsuario, EstadoPublicacion, EstadoReporte y MotivoReporte	Identifica conjuntos cerrados de valores permitidos.

Fuente: Elaboración propia.

Estereotipo <<Entity>>

El estereotipo <<Entity>> se asignó a las ocho clases principales porque representan información persistente del dominio. Las clases Favorito, Contacto, Calificacion y Reporte también son entidades, ya que almacenan datos propios de las interacciones.

Estereotipo <<enumeration>>

El estereotipo <<enumeration>> se asignó a RolUsuario, EstadoPublicacion, EstadoReporte y MotivoReporte. Estas enumeraciones permanecen como elementos independientes en el OOM y se utilizan como tipos de atributos, sin asociaciones ni cardinalidades.

Estereotipos no utilizados

Los estereotipos <<Controller>> y <<Boundary>> no se incorporaron porque el diagrama se concentra en el modelo de dominio. Agregarlos sin clases de control o interfaz reales introduciría elementos que no fueron definidos en los requisitos.

Nota técnica. En el modelo físico de SQL Server, las enumeraciones no se generan como tablas. Se implementan mediante columnas VARCHAR y reglas de validación, mientras que en el OOM se conservan como <<enumeration>>.

4.8 Validación del modelo

La validación se realizó en dos niveles. Primero se comprobó la consistencia del Modelo Orientado a Objetos mediante Check Model de PowerDesigner. Posteriormente se generó y validó el Modelo Físico de Datos, y el script SQL fue ejecutado en Microsoft SQL Server para verificar la creación de tablas, claves primarias, claves foráneas y relaciones.

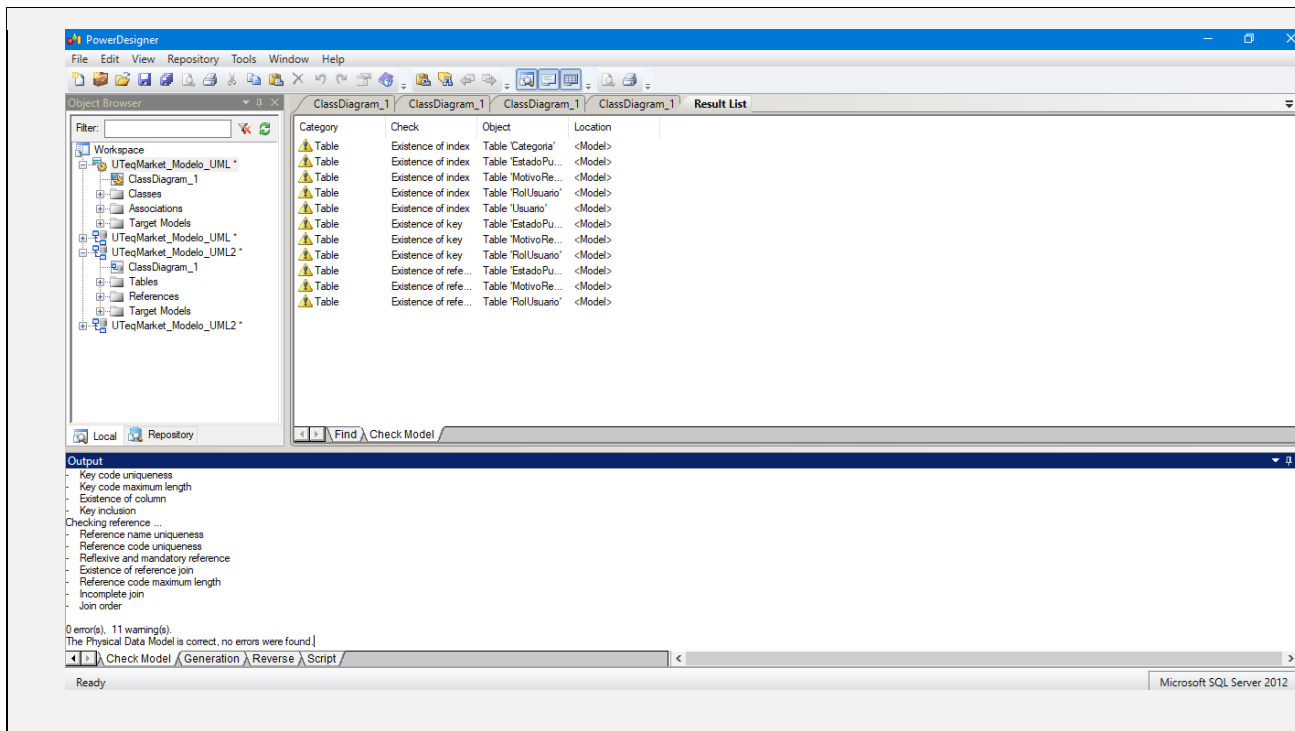


Figura 4.2. Resultado de la validación del Modelo Orientado a Objetos de UTEQMarket.

Fuente: Elaboración propia mediante PowerDesigner o SQL Server, según corresponda.

Como resultado de la comprobación del OOM, PowerDesigner registró cero errores y cero advertencias. Se verificaron los nombres y códigos de las clases, la definición de atributos y operaciones, los estereotipos, los nombres de roles y las multiplicidades de las asociaciones.

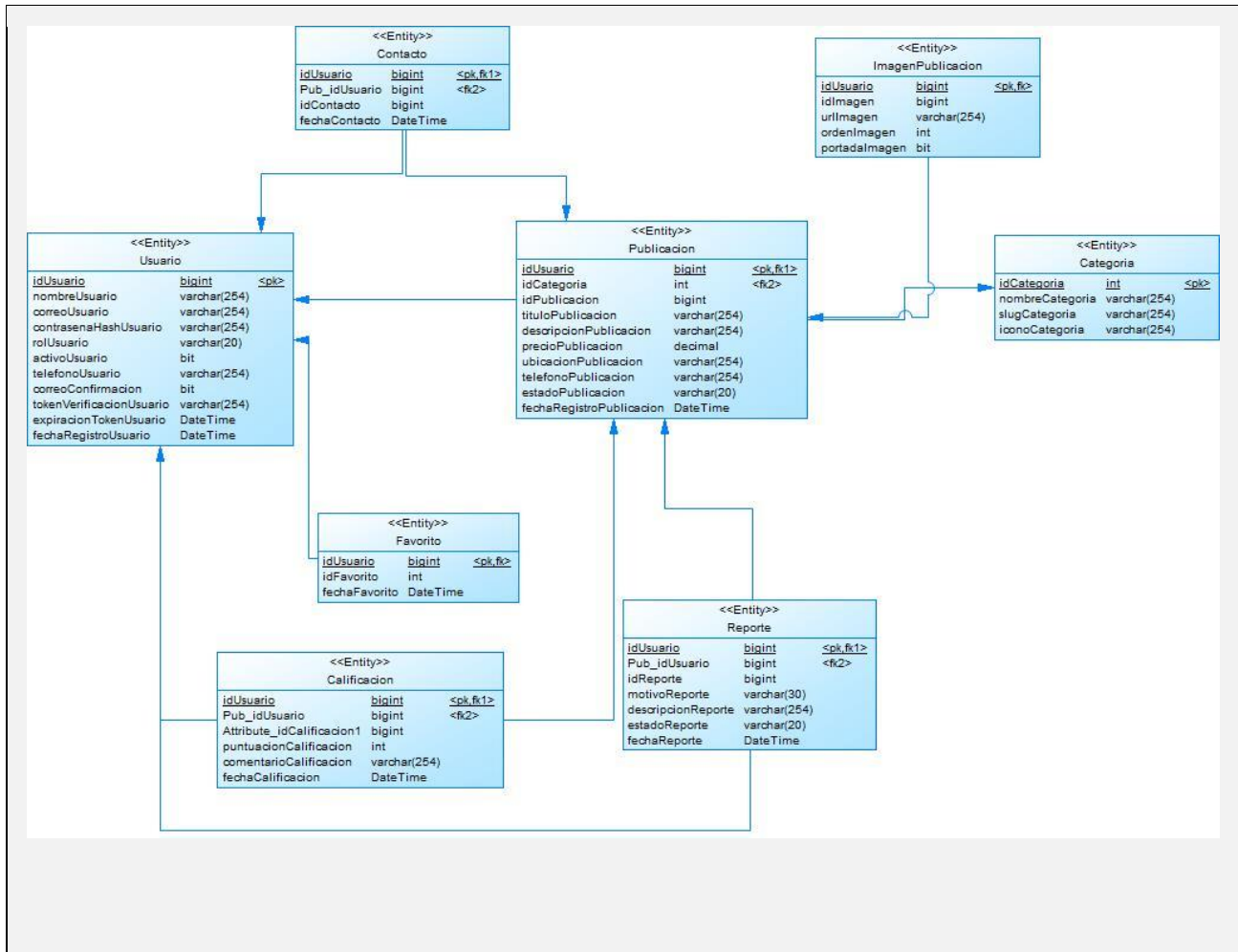


Figura 4.3. Modelo Físico de Datos final de UTEQMarket en PowerDesigner.

Fuente: Elaboración propia mediante PowerDesigner o SQL Server, según corresponda.

En el PDM se configuraron como claves primarias idUsuario, idCategoria, idPublicacion, idImagen, idFavorito, idContacto, idCalificacion e idReporte. Las asociaciones del OOM se transformaron en referencias físicas y claves foráneas. Las enumeraciones se conservaron en el modelo UML, pero en la base de datos se implementaron como columnas de texto controladas.

Tabla 4.11. Correspondencia entre asociaciones UML y claves foráneas.

Tabla principal	Tabla dependiente	Clave foránea
Usuario	Publicacion	idUsuarioPublicacion
Categoria	Publicacion	idCategoriaPublicacion
Publicacion	ImagenPublicacion	idPublicacionImagen
Usuario	Favorito	idUsuarioFavorito
Publicacion	Favorito	idPublicacionFavorito

Tabla principal	Tabla dependiente	Clave foránea
Usuario	Contacto	idUsuarioContacto
Publicacion	Contacto	idPublicacionContacto
Usuario	Calificacion	idUsuarioCalificacion
Publicacion	Calificacion	idPublicacionCalificacion
Usuario	Reporte	idUsuarioReporte
Publicacion	Reporte	idPublicacionReporte

Fuente: Elaboración propia.

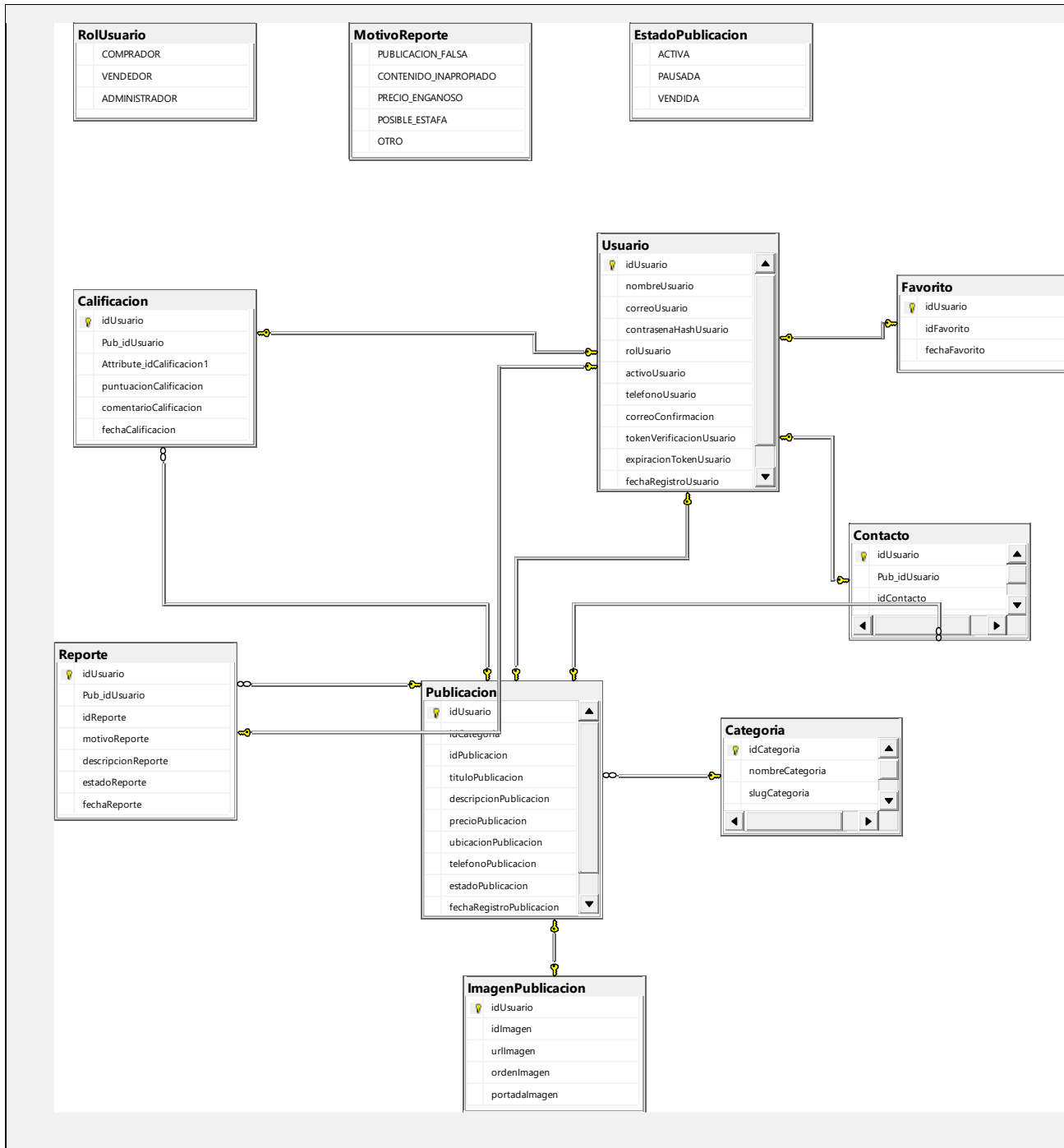


Figura 4.4. Implementación y relaciones de UTEQMarket en Microsoft SQL Server.

Fuente: Elaboración propia mediante PowerDesigner o SQL Server, según corresponda.

La ejecución del script SQL permitió crear correctamente las ocho tablas principales y sus restricciones de integridad referencial. El diagrama de SQL Server confirmó visualmente que las tablas se encuentran conectadas mediante claves foráneas y que las asociaciones no fueron transformadas en tablas innecesarias.

En conclusión, el Diagrama de Clases, el Modelo Físico de Datos y la implementación en SQL Server presentan correspondencia estructural. El modelo se considera válido y puede utilizarse como base para las siguientes etapas de diseño e implementación de UTEQMarket.

Codigo SQL server

```
CREATE DATABASE UTeqMarket_Modelo_UML
```

```
USE UTeqMarket_Modelo_UML
```

```
GO
```

```
/*=====*/
```

```
/* DBMS name:    Microsoft SQL Server 2012          */
```

```
/* Created on:   20/7/2026 0:11:32                  */
```

```
/*=====*/
```

```
if exists (select 1
```

```
    from sys.sysreferences r join sys.sysobjects o on (o.id = r.constid and o.type = 'F')
```

```
    where r.fkeyid = object_id('Calificacion') and o.name = 'FK_CALIFICA_CALIFICA_USU
```

```
ARIO')
```

```
alter table Calificacion
```

```
    drop constraint FK_CALIFICA_CALIFICA_USUARIO
```

```
go
```

```
if exists (select 1
```

```
    from sys.sysreferences r join sys.sysobjects o on (o.id = r.constid and o.type = 'F')
```



```
where      r.fkeyid      =      object_id('Calificacion')      and      o.name      =  
'FK_CALIFICA_RECIBE_CA_PUBLICAC')
```

```
alter table Calificacion
```

```
drop constraint FK_CALIFICA_RECIBE_CA_PUBLICAC
```

```
go
```

```
if exists (select 1
```

```
from sys.sysreferences r join sys.sysobjects o on (o.id = r.constid and o.type = 'F')
```

```
where      r.fkeyid      =      object_id('Contacto')      and      o.name      =  
'FK_CONTACTO_CONTACTA_USUARIO')
```

```
alter table Contacto
```

```
drop constraint FK_CONTACTO_CONTACTA_USUARIO
```

```
go
```

```
if exists (select 1
```

```
from sys.sysreferences r join sys.sysobjects o on (o.id = r.constid and o.type = 'F')
```

```
where      r.fkeyid      =      object_id('Contacto')      and      o.name      =  
'FK_CONTACTO_RECIBE_CO_PUBLICAC')
```

```
alter table Contacto
```

```
drop constraint FK_CONTACTO_RECIBE_CO_PUBLICAC
```

```
go
```

```
if exists (select 1
```

```
from sys.sysreferences r join sys.sysobjects o on (o.id = r.constid and o.type = 'F')
```

```
where r.fkeyid = object_id('Favorito') and o.name = 'FK_FAVORITO_MARCA_USUARIO')
```

```
alter table Favorito
```

```
drop constraint FK_FAVORITO_MARCA_USUARIO
```

```
go
```

```
if exists (select 1
```

```
from sys.sysreferences r join sys.sysobjects o on (o.id = r.constid and o.type = 'F')
```

```
where r.fkeyid = object_id('ImagenPublicacion') and o.name =  
'FK_IMAGENPU_CONTIENE_PUBLICAC')
```

```
alter table ImagenPublicacion
```

```
drop constraint FK_IMAGENPU_CONTIENE_PUBLICAC
```

```
go
```

```
if exists (select 1
```

```
from sys.sysreferences r join sys.sysobjects o on (o.id = r.constid and o.type = 'F')
```

```
where r.fkeyid = object_id('Publicacion') and o.name =  
'FK_PUBLICAC_CLASIFICA_CATEGORI')
```

```
alter table Publicacion
```

```
drop constraint FK_PUBLICAC_CLASIFICA_CATEGORI
```

```
go
```

```
if exists (select 1
```

```
from sys.sysreferences r join sys.sysobjects o on (o.id = r.constid and o.type = 'F')
```

```
where r.fkeyid = object_id('Publicacion') and o.name =  
'FK_PUBLICAC_PUBLICA_USUARIO')
```

```
alter table Publicacion
```

```

drop constraint FK_PUBLICAC_PUBLICA_USUARIO
go

if exists (select 1
    from sys.sysreferences r join sys.sysobjects o on (o.id = r.constid and o.type = 'F')
    where      r.fkeyid      =      object_id('Reporte')      and      o.name      =
'FK_REPORTE_ES_REPORT_PUBLICAC')
alter table Reporte
    drop constraint FK_REPORTE_ES_REPORT_PUBLICAC
go

if exists (select 1
    from sys.sysreferences r join sys.sysobjects o on (o.id = r.constid and o.type = 'F')
    where r.fkeyid = object_id('Reporte') and o.name = 'FK_REPORTE_REPORTA_USUARIO')
alter table Reporte
    drop constraint FK_REPORTE_REPORTA_USUARIO
go

if exists (select 1
    from sysindexes
    where id  = object_id('Calificacion')
    and  name = 'CALIFICA_FK'
    and  indid > 0
    and  indid < 255)

```

```
drop index Calificacion.CALIFICA_FK  
go
```

```
if exists (select 1  
           from sysobjects  
           where id = object_id('Calificacion')  
           and type = 'U')  
drop table Calificacion  
go
```

```
if exists (select 1  
           from sysobjects  
           where id = object_id('Categoria')  
           and type = 'U')  
drop table Categoria  
go
```

```
if exists (select 1  
           from sysindexes  
           where id = object_id('Contacto')  
           and name = 'CONTACTA_FK'  
           and indid > 0  
           and indid < 255)  
drop index Contacto.CONTACTA_FK
```

```
go
```

```
if exists (select 1  
           from sysobjects  
           where id = object_id('Contacto')  
           and type = 'U')  
drop table Contacto
```

```
go
```

```
if exists (select 1  
           from sysobjects  
           where id = object_id('EstadoPublicacion')  
           and type = 'U')  
drop table EstadoPublicacion
```

```
go
```

```
if exists (select 1  
           from sysindexes  
           where id = object_id('Favorito')  
           and name = 'MARCA_FK'  
           and indid > 0  
           and indid < 255)  
drop index Favorito.MARCA_FK
```

```
go
```

```
if exists (select 1
          from sysobjects
          where id = object_id('Favorito')
          and type = 'U')
drop table Favorito
go
```

```
if exists (select 1
          from sysindexes
          where id = object_id('ImagenPublicacion')
          and name = 'CONTIENE_FK'
          and indid > 0
          and indid < 255)
drop index ImagenPublicacion.CONTIENE_FK
go
```

```
if exists (select 1
          from sysobjects
          where id = object_id('ImagenPublicacion')
          and type = 'U')
drop table ImagenPublicacion
go
```

```
if exists (select 1
          from sysobjects
          where id = object_id('MotivoReporte')
          and type = 'U')
drop table MotivoReporte
go
```

```
if exists (select 1
          from sysindexes
          where id = object_id('Publicacion')
          and name = 'PUBLICA_FK'
          and indid > 0
          and indid < 255)
drop index Publicacion.PUBLICA_FK
go
```

```
if exists (select 1
          from sysobjects
          where id = object_id('Publicacion')
          and type = 'U')
drop table Publicacion
go
```

```
if exists (select 1
```

```
from sysindexes
where id = object_id('Reporte')
and name = 'REPORTA_FK'
and indid > 0
and indid < 255)
drop index Reporte.REPORTA_FK
go
```

```
if exists (select 1
from sysobjects
where id = object_id('Reporte')
and type = 'U')
drop table Reporte
go
```

```
if exists (select 1
from sysobjects
where id = object_id('RolUsuario')
and type = 'U')
drop table RolUsuario
go
```

```
if exists (select 1
from sysobjects
```



```

        where id = object_id('Usuario')
        and type = 'U')

drop table Usuario

go

/*=====*/
/* Table: Calificacion */
/*=====*/

create table Calificacion (
    idUsuario      bigint      not null,
    Pub_idUsuario  bigint      null,
    Attribute_idCalificacion1 bigint      null,
    puntuacionCalificacion int      null,
    comentarioCalificacion varchar(254)    null,
    fechaCalificacion DateTime      null,
    constraint PK_CALIFICACION primary key (idUsuario)
)

go

/*=====*/
/* Index: CALIFICA_FK */
/*=====*/

create index CALIFICA_FK on Calificacion (
    idUsuario ASC

```

)

go

/*=====*/

/* Table: Categoria */

/*=====*/

create table Categoria (

idCategoria int not null,

nombreCategoria varchar(254) null,

slugCategoria varchar(254) null,

iconoCategoria varchar(254) null,

constraint PK_CATEGORIA primary key (idCategoria)

)

go

/*=====*/

/* Table: Contacto */

/*=====*/

create table Contacto (

idUsuario bigint not null,

Pub_idUsuario bigint null,

idContacto bigint null,

fechaContacto DateTime null,

constraint PK_CONTACTO primary key (idUsuario)

```

)
go

/*=====*/

/* Index: CONTACTA_FK */

/*=====*/

create index CONTACTA_FK on Contacto (
idUsuario ASC
)
go

/*=====*/

/* Table: EstadoPublicacion */

/*=====*/

create table EstadoPublicacion (
    ACTIVA      int      null,
    PAUSADA     int      null,
    VENDIDA     int      null
)
go

/*=====*/

/* Table: Favorito */

/*=====*/

```

```

create table Favorito (
    idUsuario      bigint      not null,
    idFavorito     int         null,
    fechaFavorito  DateTime    null,
    constraint PK_FAVORITO primary key (idUsuario)
)
go

```

```

/*=====*/
/* Index: MARCA_FK                                */
/*=====*/

create index MARCA_FK on Favorito (
    idUsuario ASC
)
go

```

```

/*=====*/
/* Table: ImagenPublicacion                        */
/*=====*/

create table ImagenPublicacion (
    idUsuario      bigint      not null,
    idImagen       bigint      null,
    urlImagen      varchar(254) null,
    ordenImagen    int         null,

```

```

    portadaImagen    bit            null,

    constraint PK_IMAGENPUBLICACION primary key (idUserario)
)
go

/*=====*/
/* Index: CONTIENE_FK                                */
/*=====*/

create index CONTIENE_FK on ImagenPublicacion (
idUserario ASC
)
go

/*=====*/
/* Table: MotivoReporte                                */
/*=====*/

create table MotivoReporte (
    PUBLICACION_FALSA    int            null,
    CONTENIDO_INAPROPIADO int            null,
    PRECIO_ENGANOSO      int            null,
    POSIBLE_ESTAFA       int            null,
    OTRO                  int            null
)
go

```

```

/*=====*/

/* Table: Publicacion */

/*=====*/

create table Publicacion (

    idUsuario      bigint      not null,

    idCategoria    int         null,

    idPublicacion  bigint      null,

    tituloPublicacion varchar(254) null,

    descripcionPublicacion varchar(254) null,

    precioPublicacion decimal    null,

    ubicacionPublicacion varchar(254) null,

    telefonoPublicacion varchar(254) null,

    estadoPublicacion varchar(20) null,

    fechaRegistroPublicacion DateTime null,

    constraint PK_PUBLICACION primary key (idUsuario)

)

go

/*=====*/

/* Index: PUBLICA_FK */

/*=====*/

create index PUBLICA_FK on Publicacion (

idUsuario ASC

```

```

)
go

/*=====*/

/* Table: Reporte */

/*=====*/

create table Reporte (
    idUsuario      bigint      not null,
    Pub_idUsuario  bigint      null,
    idReporte      bigint      null,
    motivoReporte  varchar(30)  null,
    descripcionReporte  varchar(254)  null,
    estadoReporte  varchar(20)  null,
    fechaReporte   DateTime     null,
    constraint PK_REPORTA primary key (idUsuario)
)
go

/*=====*/

/* Index: REPORTA_FK */

/*=====*/

create index REPORTA_FK on Reporte (
    idUsuario ASC
)

```

go

```
/*=====*/
```

```
/* Table: RolUsuario */
```

```
/*=====*/
```

```
create table RolUsuario (  
    COMPRADOR      int      null,  
    VENDEDOR        int      null,  
    ADMINISTRADOR   int      null  
)
```

go

```
/*=====*/
```

```
/* Table: Usuario */
```

```
/*=====*/
```

```
create table Usuario (  
    idUsuario      bigint      not null,  
    nombreUsuario   varchar(254)  null,  
    correoUsuario   varchar(254)  null,  
    contrasenaHashUsuario varchar(254)  null,  
    rolUsuario      varchar(20)   null,  
    activoUsuario   bit           null,  
    telefonoUsuario varchar(254)  null,  
    correoConfirmacion bit        null,
```



```
tokenVerificacionUsuario varchar(254)    null,  
expiracionTokenUsuario DateTime        null,  
fechaRegistroUsuario DateTime          null,  
constraint PK_USUARIO primary key (idUserio)  
)  
go
```

```
alter table Calificacion  
  
add constraint FK_CALIFICA_CALIFICA_USUARIO foreign key (idUserio)  
references Usuario (idUserio)  
go
```

```
alter table Calificacion  
  
add constraint FK_CALIFICA_RECIBE_CA_PUBLICAC foreign key (Pub_idUsuario)  
references Publicacion (idUserio)  
go
```

```
alter table Contacto  
  
add constraint FK_CONTACTO_CONTACTA_USUARIO foreign key (idUserio)  
references Usuario (idUserio)  
go
```

```
alter table Contacto  
  
add constraint FK_CONTACTO_RECIBE_CO_PUBLICAC foreign key (Pub_idUsuario)
```

```
references Publicacion (idUserario)
```

```
go
```

```
alter table Favorito
```

```
add constraint FK_FAVORITO_MARCA_USUARIO foreign key (idUserario)
```

```
references Usuario (idUserario)
```

```
go
```

```
alter table ImagenPublicacion
```

```
add constraint FK_IMAGENPU_CONTIENE_PUBLICAC foreign key (idUserario)
```

```
references Publicacion (idUserario)
```

```
go
```

```
alter table Publicacion
```

```
add constraint FK_PUBLICAC_CLASIFICA_CATEGORI foreign key (idCategoria)
```

```
references Categoria (idCategoria)
```

```
go
```

```
alter table Publicacion
```

```
add constraint FK_PUBLICAC_PUBLICA_USUARIO foreign key (idUserario)
```

```
references Usuario (idUserario)
```

```
go
```

```
alter table Reporte
```

```
add constraint FK_REPORTE_ES_REPORT_PUBLICAC foreign key (Pub_idUsuario)
references Publicacion (idUsuario)

go
```

```
alter table Reporte
```

```
add constraint FK_REPORTE_REPORTA_USUARIO foreign key (idUsuario)
references Usuario (idUsuario)

go
```